

scripts/ci.sh — User Guide

Last verified: 2026-08-26 (§6.J corpus floors — hermetic-suite no-ops and a device-less --full no longer report a pass; LVA vacuous-pass sweep F9 + F21) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate)

Overview

Stub doc generated from the script's in-source header comment. See scripts/ci.sh for canonical behavior. This stub exists so the CM-SCRIPT-DOCS-SYNC pre-push gate (.githubhooks/pre-push Check 9) starts gating modifications to this script.

The script's own header documentation (verbatim):

scripts/ci.sh – local-only CI gate for Lava.

Per the Local-Only CI/CD constitutional rule, this script IS the project's CI/CD apparatus. The same script runs in three modes:

- | | |
|----------------|---|
| --changed-only | Fast subset for the pre-push hook (Spotless, unit tests of changed modules, constitutional doc parser, forbidden-files check). No real-device tests; no mutation tests. |
| --full | All gates – unit tests across every module, parity gate, mutation tests where wired, fixture freshness, Compose UI Challenge Tests (requires a connected Android device or emulator). Used at tag time. |
| (default) | Same as --full. |

Per Sixth Law clause 5: passing CI is necessary, NOT sufficient for a release. The operator real-device verification per Task 5.22 of SP-3a is the load-bearing acceptance gate; this script certifies the codebase is shippable, not that the user-visible feature is shipped.

Usage

See the script's in-source comment block (above) for canonical usage examples.

Pipeline steps (current order)

1. Hosted-CI forbidden-files check (Local-Only CI/CD rule)
2. Host-power forbidden-command regex check (Host Stability rule)
3. Spotless (./gradlew spotlessCheck)
4. Unit tests across changed modules
5. Constitutional doc parser (scripts/check-constitution.sh)
6. **5a1 (added 2026-05-14):** §6.AC non-fatal-coverage scan (STRICT default; closes §6.AC-debt + CM-NONFATAL-COVERAGE gate)
7. **5a2 (added 2026-05-15):** §6.AB Challenge-discrimination scan (STRICT default; closes §6.AB-debt + CM-CHALLENGE-DISCRIMINATION gate)
8. **5a3 (added 2026-05-15, 31st §6.L):** §6.AE per-feature Challenge coverage scan (STRICT default after per-feature backfill drained the queue 2026-05-15; CM-CHALLENGE-COVERAGE gate; LAVA_CHALLENGE_COVERAGE_STRICT=0 to revert to advisory)
9. **5a4 (added 2026-05-15, constitution-compliance plan Phase 1):** §11.4.32 verify-all-constitution-rules sweep (STRICT default; the §11.4.32 mandated enforcement engine for every other constitution rule; wraps every individual gate above + every hermetic test suite into one invocation; produces per-run attestation JSON at .lava-ci-evidence/verify-all/<UTC-timestamp>.json; CM-VERIFY-ALL-CONSTITUTION-RULES gate)
10. 5b. Hermetic bash test suites (under tests/)
11. (Full-mode only) Parity, mutation, fixture-freshness, Compose UI Challenge Tests on connected device

Forbidden-files check scoped to git ls-files — added 2026-08-20

Fixes a real false-positive in Pipeline step 1: the check previously used a raw `find .` over the working directory, which also walked (a) other worktrees checked out under `.claude/worktrees/agent-*/` — each is a DIFFERENT branch's file tree, physically nested under this repo's working directory by the harness but irrelevant to what the CURRENT branch is about to push — and (b) vendored third-party content inside nested submodules-of-submodules (e.g. HelixQA's `tools/opensource/*`, which legitimately ships upstream `.github/workflows/*` files as that nested repo's own concern). Both classes produced spurious FORBIDDEN HOSTED-CI FILES rejections on a push whose own tracked tree contained zero

such files. Fixed by scoping the scan to `git ls-files` (this branch's actually-tracked files only) — submodules naturally excluded as gitlinks (not descended into without `--recurse-submodules`, which is deliberately not passed; submodule-internal governance is scoped separately per §6.F), other worktrees naturally excluded since their files aren't part of this branch's tree at all.

Bluff-Audit: `scripts/ci.sh` Hosted-CI forbidden-files check Mutation: `git add -f a real .github/workflows/mutation-test.yml` fixture into the tracked tree, confirming the NEW `git ls-files`-scoped check still correctly DETECTS a genuine violation (true-positive preserved). Observed-Failure: N/A for this direction (positive detection confirmed); the OLD `find .`-based check was separately confirmed to false-positive on real, pre-existing, untracked content under `.claude/worktrees/` and `submodules/helixqa/tools/opensource/` during an actual `git push` attempt against this exact commit range — that rejection is the falsifiability evidence for the bug this fix closes. Reverted: yes — the mutation fixture was `git reset + rm -rf`'d immediately after the positive-detection check passed; confirmed via `git status --short` showing a clean tree before this commit.

2026-08-26 update — §6.J corpus floor on the hermetic suite loop (finding F9)

The defect

The hermetic-suite loop contained **three silent no-ops**, and each one turned a *failing* suite into a clean run — the loop reported success having executed nothing. Measured against a tree in which `tests/pre-push/check9_test.sh` genuinely exits 1:

Case	Result
Baseline (<code>check9_test.sh</code> fails)	exit 1 — correct
Mutation A: delete only <code>tests/pre-push/check4_test.sh</code> ; <code>check9_test.sh</code> still present, still failing	loop completed, exit 0
Mutation B: <code>chmod -x tests/firebase/run_all.sh</code> (runner exits 1)	exit 0
Mutation C: delete the whole <code>tests/check-constitution/</code> directory	exit 0

Mutation A is the sharpest. The flat-layout branch keyed on a single sentinel filename (`if [[ -f "$suite_dir/check4_test.sh" ]]`), so deleting an *unrelated* file disabled an entire failing suite. Mutation B and C are the same class arriving by different routes:

a non-executable runner fell through every branch, and an absent directory was skipped by the outer `if [[ -d ... ]]` with no record that it was skipped.

`scripts/verify-all-constitution-rules.sh` had already fixed exactly this pattern in its own loops — an absent suite becomes a FAIL row rather than disappearing. `ci.sh` carried the unfixed copy.

What the loop now does

The declared suite list is explicit (`HERMETIC_SUITE_DIRS`), and each of the three conditions is now an **explicit failure** rather than a skip:

- **Directory absent** → recorded as DIRECTORY ABSENT (the suite did not run; before this floor it was skipped in silence).
- **run_all.sh exists but is not executable** → recorded as EXISTS BUT IS NOT EXECUTABLE (`chmod +x ...`); the suite did not run.
- **Flat-layout glob matched nothing** → recorded as NO runner and NO `check*_test.sh` matched.

The flat-layout branch is now keyed on the **glob**, never on one sentinel filename — Mutation A is precisely what a sentinel key produces. Individual test-file failures are collected rather than aborting, so one run names every broken suite instead of only the first.

Each flat-layout branch additionally compares the number of test files it executed against what `git ls-files` declares for that directory, and reports a shortfall as `executed <n> test file(s) but the git index declares <m>`: working-tree drift, not a smaller suite. The expectation is derived, never hardcoded, so adding or removing a suite cannot silently lower the bar.

Failure output states `Examined: <n> of <m> declared suite(s)` and spells out the principle: *a suite that is absent, non-executable, or whose glob matched nothing did NOT pass — it did not run*. A final floor refuses when **zero** suites executed. On success the loop prints `<n>/<m>` hermetic suites executed, so the count is a reported fact.

2026-08-26 update — --full no longer certifies a run whose device gate never ran (finding F21)

The defect

`--full` printed `==> All gates passed and exited 0` with the Compose UI Challenge Tests skipped whenever no connected device was detected. The skip was announced on `stdout` — but the **machine-**

readable output was byte-identical either way: the evidence directory recorded only `mode` and `sha`, with no field saying whether the device gate ran.

A consumer reading that directory could not distinguish a full run from a device-less one. `mode=--full` was therefore a claim the evidence did not support — and `scripts/tag.sh` consumes this directory as proof of a full run.

What changed

Two changes, deliberately paired:

1. **The verdict is recorded.** `$EVIDENCE_DIR/device_tests` is written on **both** exit paths — ran or skipped — so a consumer never has to infer the verdict from the absence of a file. `--changed-only` writes skipped explicitly: it declares up front that it does not run the device gate, and the field states that as a fact.
2. **A `--full` run whose device gate did not run is no longer reported as a full pass.** It exits **1** with `CI_GATE_INCOMPLETE`, and the message distinguishes the cause: *this is not a test failure — the tests did not run at all, which is why reporting "All gates passed" here would be a §6.J bluff*. The remedy it prints is to bring up a §6.AH-conformant container/VM emulator via `scripts/run-challenge-matrix.sh` (never host-direct, never a live ADB device), or to run `--changed-only` if a device-less subset is genuinely what was wanted.

There is deliberately **no bypass flag**. §6.Z clause 6 forbids one, and a flag that converts "did not run" into "passed" is the exact bluff this floor exists to close.

On success the final line now names the verdict it recorded: `==> All gates passed (device_tests=ran)`.

Consequence for callers

A `--full` invocation on a host with no emulator, which previously exited 0, now exits 1. That is the intended outcome. Use `--changed-only` for the device-less subset, or bring up the container/VM emulator first.

Evidence directory contents (updated)

File	Meaning
<code>mode</code>	<code>--changed-only</code> or <code>--full</code>
<code>sha</code>	git rev-parse HEAD at the time of the run
<code>device_tests</code>	

File	Meaning
	ran or skipped — written on every exit path (added 2026-08-26)

§6.AE matrix gate (separate entry point)

For §6.AE.2 gate-mode runs producing per-AVD attestation: bash scripts/run-challenge-matrix.sh. The runner correctly REFUSES to claim success on hosts lacking KVM (darwin/arm64) — exits 2 with a host-gap diagnostic. Real attestations require a Linux x86_64 + KVM gate-host.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation **MUST** stay in sync with the codebase — no doc may be out of sync with its script.

Cross-references

- scripts/ci.sh — the script itself
- docs/helix-constitution-gates.md — gate inventory
- HelixConstitution Constitution.md §11.4.18 (the mandate)
- Lava CLAUDE.md §6.AD (HelixConstitution Inheritance)